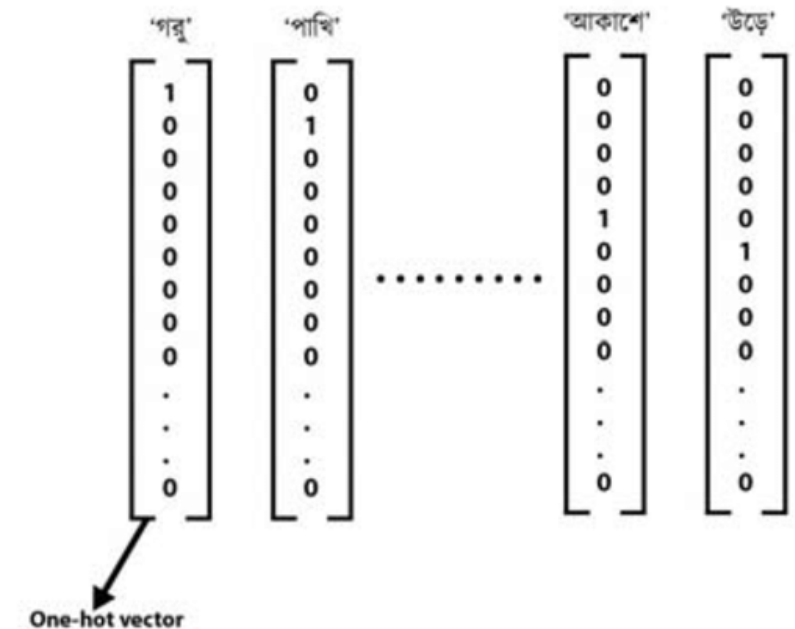


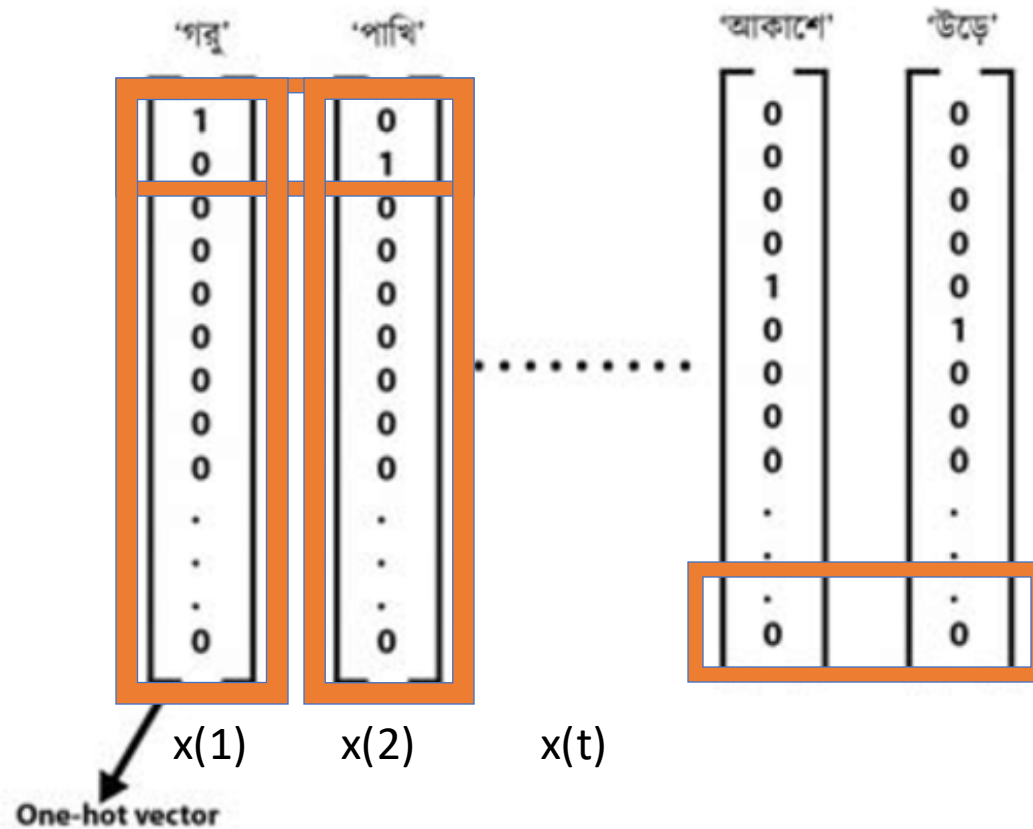
S9: Deep Learning for Text

Full order information

- Jinyang and the TA love AUDers
- AUDers love Jinyang and the TA
- Order information is very important
 - Machine translation
 - Chatbot
 - Sentiment analysis
- How to represent text to keep the order information fully?
 - Index
 - One-hot
 - Word-embedding
- How to model to learn such ordered info?

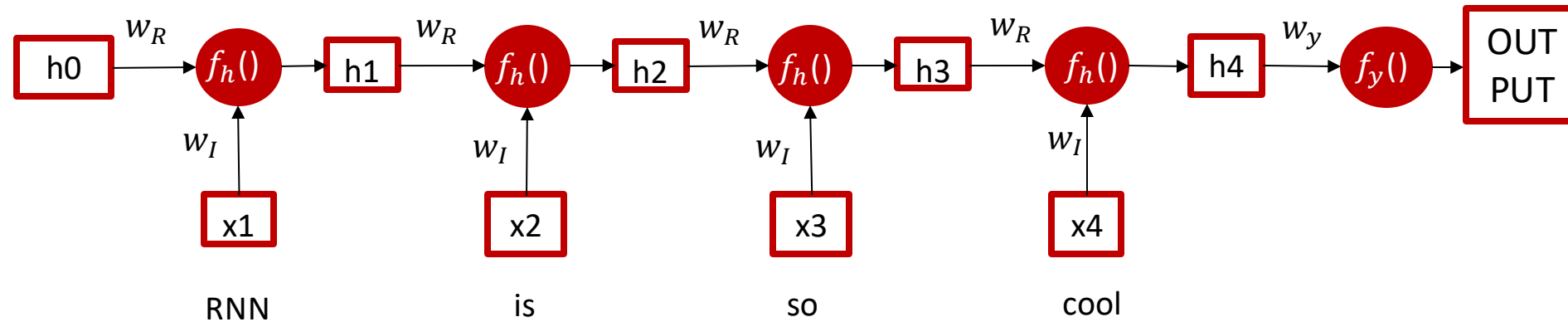


Temporal or sequential structures

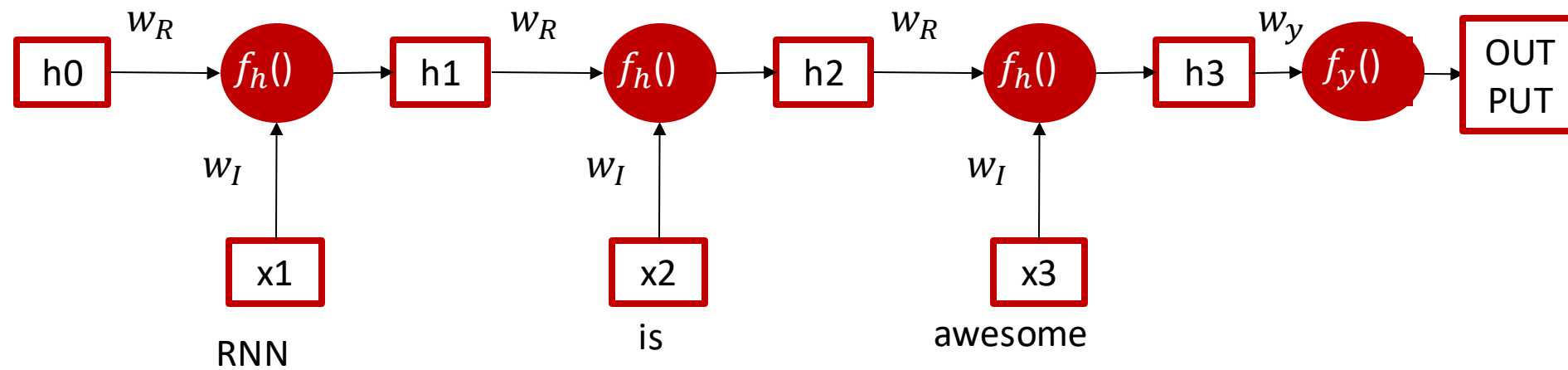


Recurrent Neural Networks

RNN is so cool



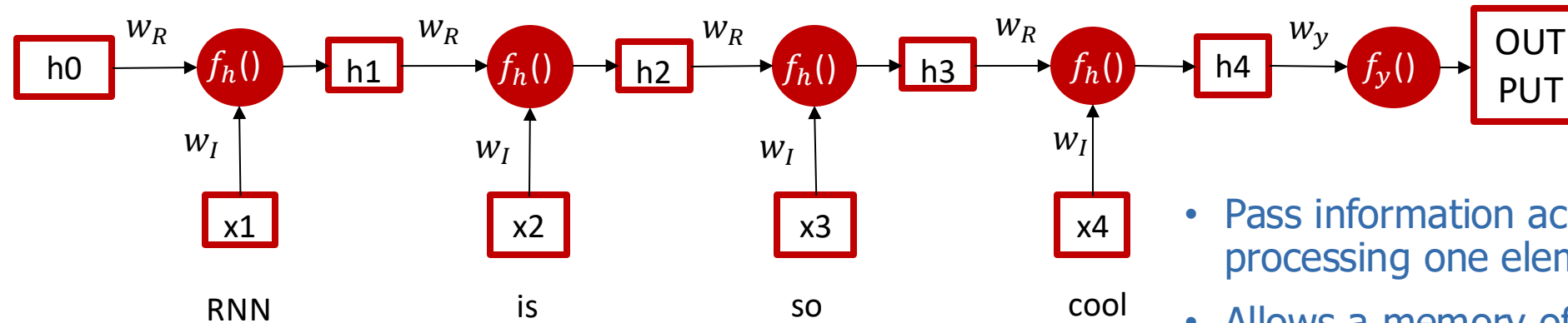
RNN is awesome



$$h(t) = f_h(w_R * h(t-1) + w_I * x(t))$$

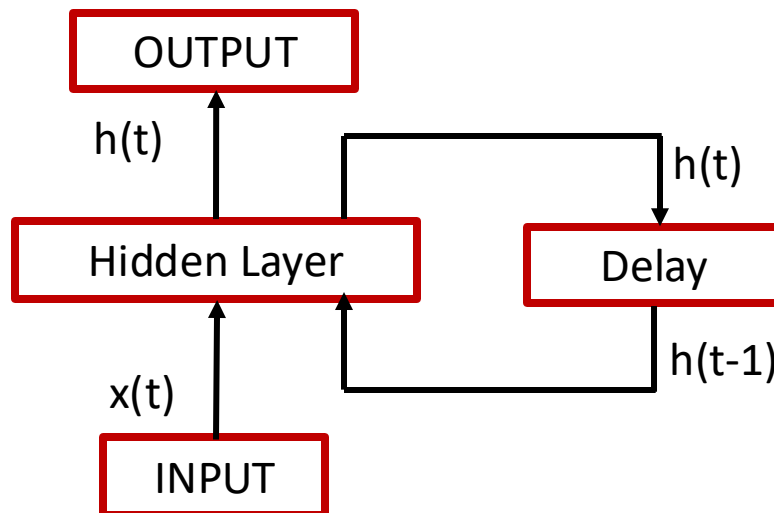
Recurrent Neural Networks (Rolled)

RNN is so cool



- Pass information across sequential steps, processing one element/feature at a step/time
- Allows a memory of the previous inputs to persist in the model's internal state and influence the outcome
- Exhibit temporal dynamic behaviour for a time sequence

$$h(t) = f_h(w_R * h(t-1) + w_I * x(t))$$



Problem with vanishing or exploding gradients

- Don't learn long term dependencies.
- CIS434ers are hard workers.
- VS.
- CIS434ers, students of JZ, are hard workers.

BAD!!!!
Misguided!!!!
Unacceptable!!!!

The vanishing gradient problem

- The weights updates following

$$W \leftarrow W - \alpha \frac{\partial E_t}{\partial W_R}$$
$$\frac{\partial E_t}{\partial W_R} = \sum_{i=0}^t \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \boxed{\frac{\partial h_t}{\partial h_i}} \frac{\partial h_i}{\partial W_R}$$

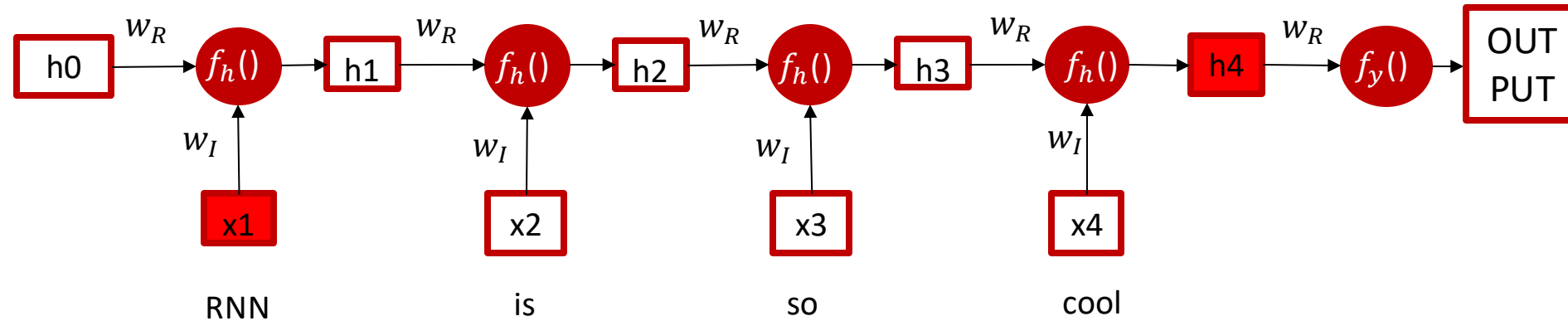
- RNN's use back propagation, back propagation uses chain rule, chain rule multiplies derivatives

$$\frac{\partial h_k}{\partial h_i} = \prod_i^k \text{diag}(f'(W_I x_i + W_R h_{i-1})) W_R$$

- If W_R are between 0 and 1 the product vanishes as the chain gets longer, or the product explodes if greater than 1
- For $i \ll t$, will W vary for different term x_i ? In other words, will term x_i affect W ?

The vanishing gradient problem

RNN is so cool

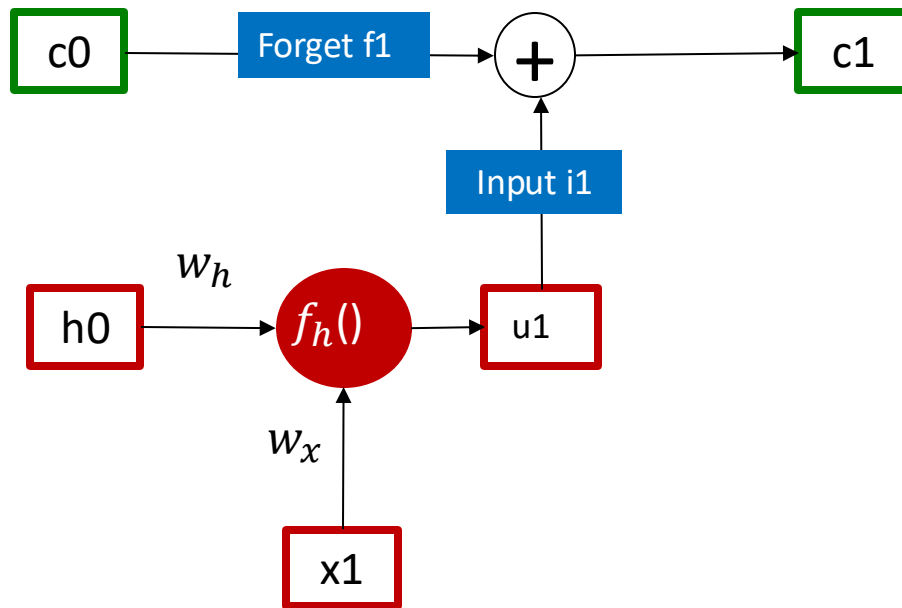


For $i \ll t$, will W vary for different term x_i ? In other words, will term x_i affect W ?

Long Short Term Memory (LSTM)

- LSTM provide solution to the vanishing/exploding gradient problem.
- Solution: **Memory Cell**, which is updated at each step in the sequence.
- Three Gates control the flow of information to and from the Memory cell
 - **Input Gate**: protect the current step from irrelevant inputs
 - **Output Gate**: prevents current step from passing irrelevant information to later steps.
 - **Forget Gate**: limits information passed from one cell to the next.

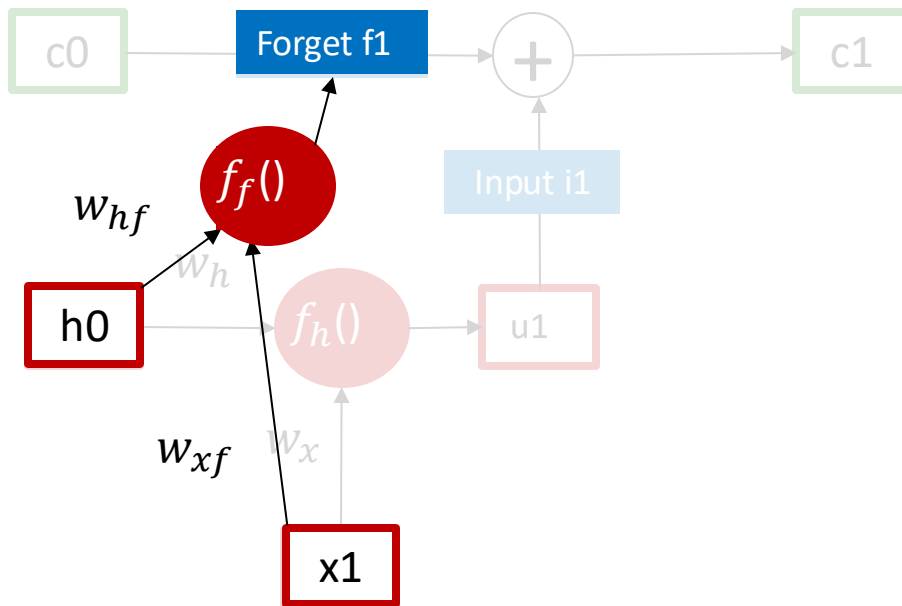
LSTM



$$c_1 = f_1 \cdot c_0 + i_1 \cdot u_1$$

$$c_t = f_t \cdot c_{t-1} + i_t \cdot u_t$$

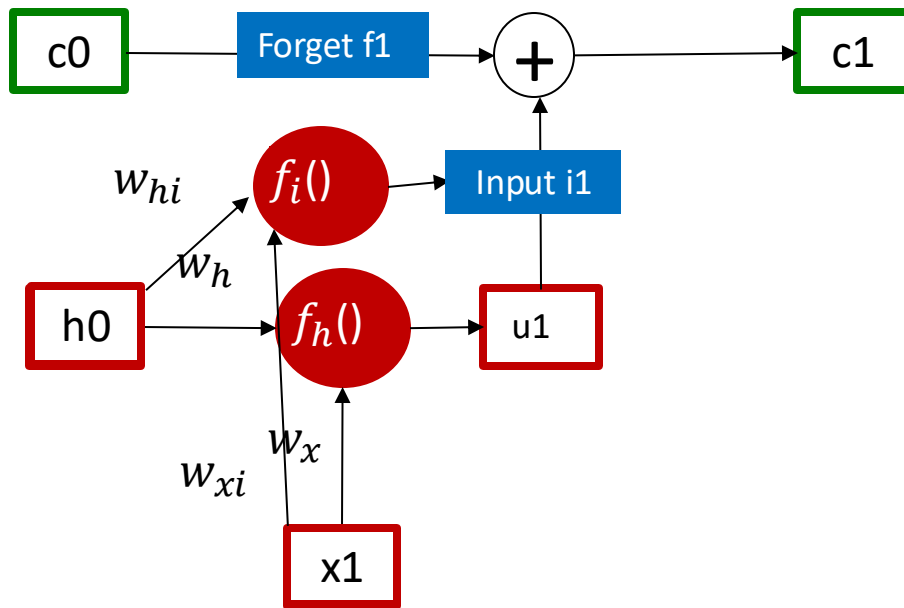
LSTM



$$f_1 = f_f(W_{hf} * h_0 + W_{xf} * x_1)$$

$$f_t = f_f(W_{hf} * h_{t-1} + W_{xf} * x_t)$$

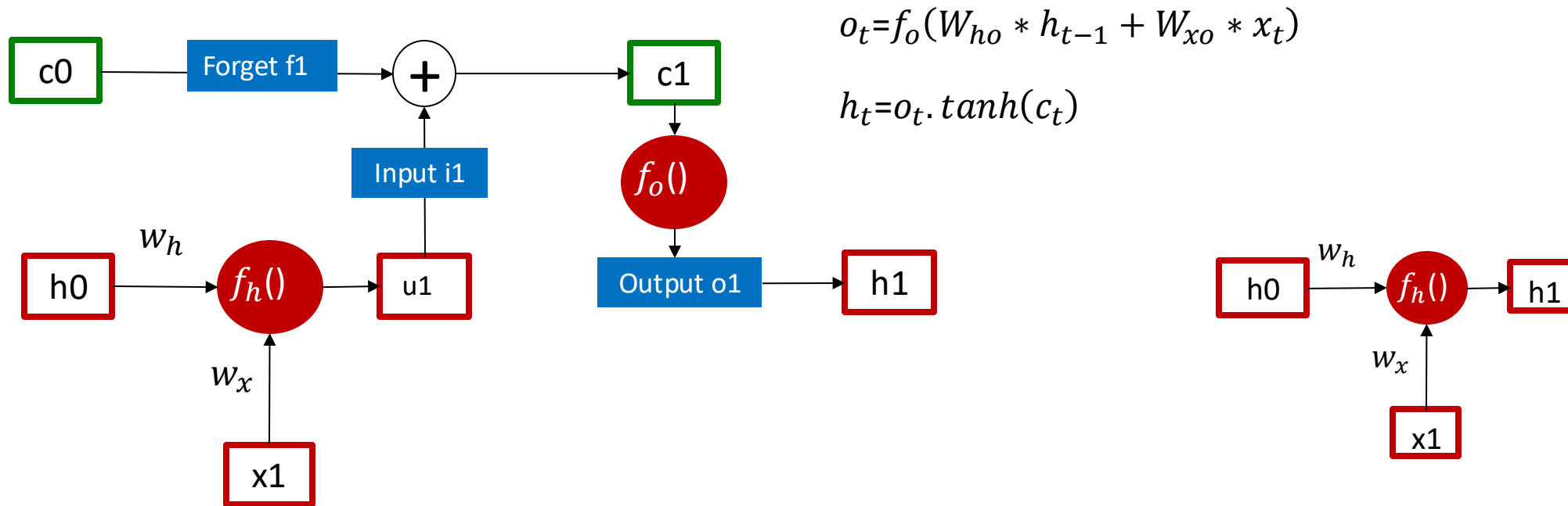
LSTM



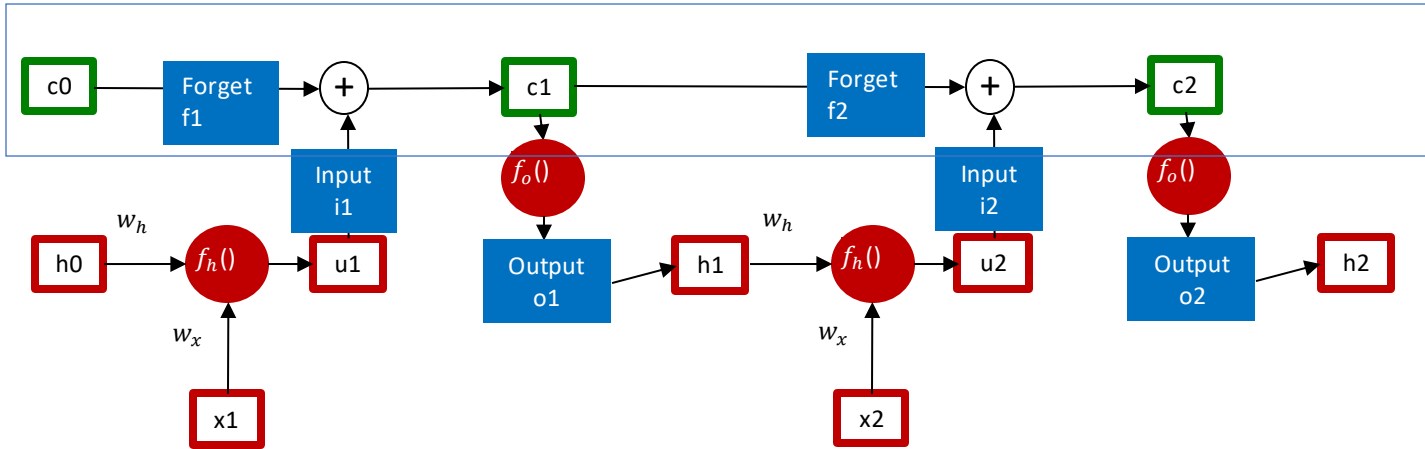
$$i_1 = f_i(W_{hi} * h_0 + W_{xi} * x_1)$$

$$i_t = f_f(W_{hi} * h_{t-1} + W_{xi} * x_t)$$

LSTM



LSTM (Unrolled)

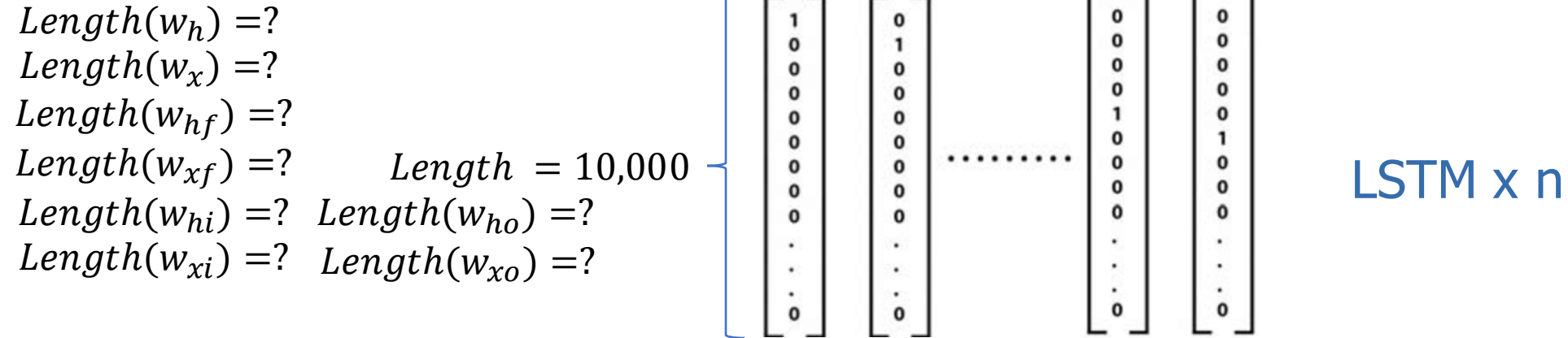


- The error gradient $\frac{\partial E_t}{\partial W_R}$ is also a function of $\frac{\partial C_t}{\partial C_i}$ for $i=1...t$

- $\frac{\partial C_k}{\partial C_i} = \prod_i^k \frac{\partial C_t}{\partial C_{t-1}}$

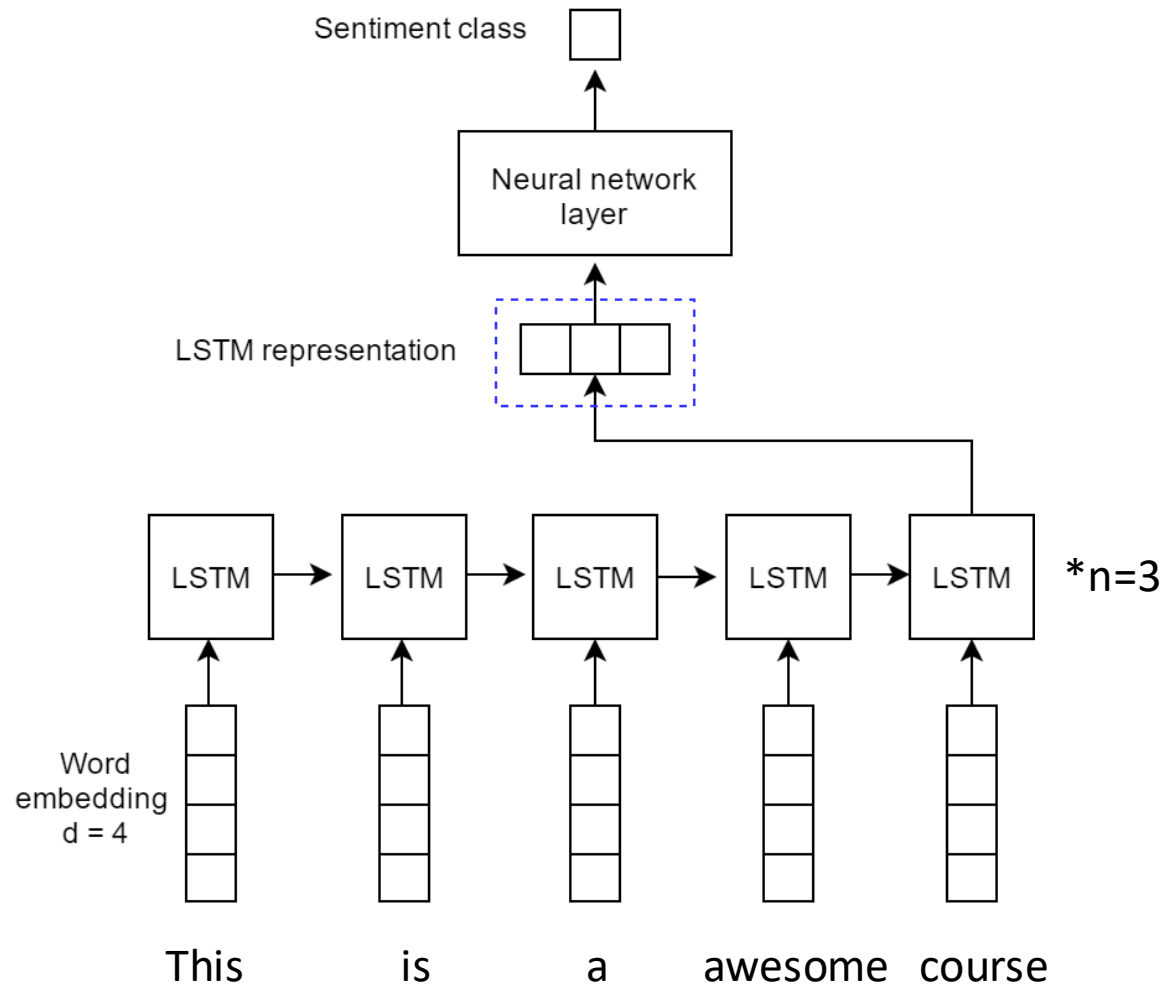
- where
$$\begin{aligned} \frac{\partial C_t}{\partial C_{t-1}} = & C_{t-1} \sigma'(\cdot) W_f * o_{t-1} \tanh'(C_{t-1}) \\ & + \tilde{C}_t \sigma'(\cdot) W_i * o_{t-1} \tanh'(C_{t-1}) \\ & + i_t \tanh'(\cdot) W_C * o_{t-1} \tanh'(C_{t-1}) \\ & + f_t \end{aligned}$$

Dimensionality problem



- Reduction: embedding layer
- Word embedding: map a one-hot vector (or an index) to a vector of real values
 - 'Good' – 86 – $[0,0,0,0, \dots, 0,1,0, \dots, 0] - [0.21,0.97,0.35]$
- Embedding layer: map a one-hot vector (or an index) to a vector of unknown parameters
 - 'Good' – 86 – $[0,0,0,0, \dots, 0,1,0, \dots, 0] - [b_1,b_2,b_3,\dots]$

LSTM architecture for sentiment analysis



Python: LSTM for subjectivity/objectivity

```
import numpy as np
from nltk.corpus import subjectivity

n_instances = 5000
subj_docs = [(sent, 'subj') for sent in
subjectivity.sents(categories='subj')[:n_instances]]
obj_docs = [(sent, 'obj') for sent in
subjectivity.sents(categories='obj')[:n_instances]]

docs = subj_docs+obj_docs
docs_x = [i[0] for i in docs]
docs_c = [i[1] for i in docs]

# define vocab
from collections import Counter
words = [j for i in docs_x for j in i]
count_words = Counter(words)
total_words = len(words)
sorted_words = count_words.most_common(total_words)
vocab_to_int = {w: i+1 for i, (w,c) in enumerate(sorted_words)}# note 0 is for null
```

Python: LSTM for subjectivity/objectivity

```
# index encoding
text_int = []
for i in docs_x:
    r = [vocab_to_int[w] for w in i]
    text_int.append(r)

# dummy label
encoded_labels = [1 if label == 'subj' else 0 for label in docs_c]

# split test and training
y_train = np.array(encoded_labels[:4000] + encoded_labels[5000:9000])
y_test = np.array(encoded_labels[4001:5001] + encoded_labels[9000::])
x_train = np.array(text_int[:4000] + text_int[5000:9000])
x_test = np.array(text_int[4001:5001] + text_int[9000::])
```

Python: LSTM for subjectivity/objectivity

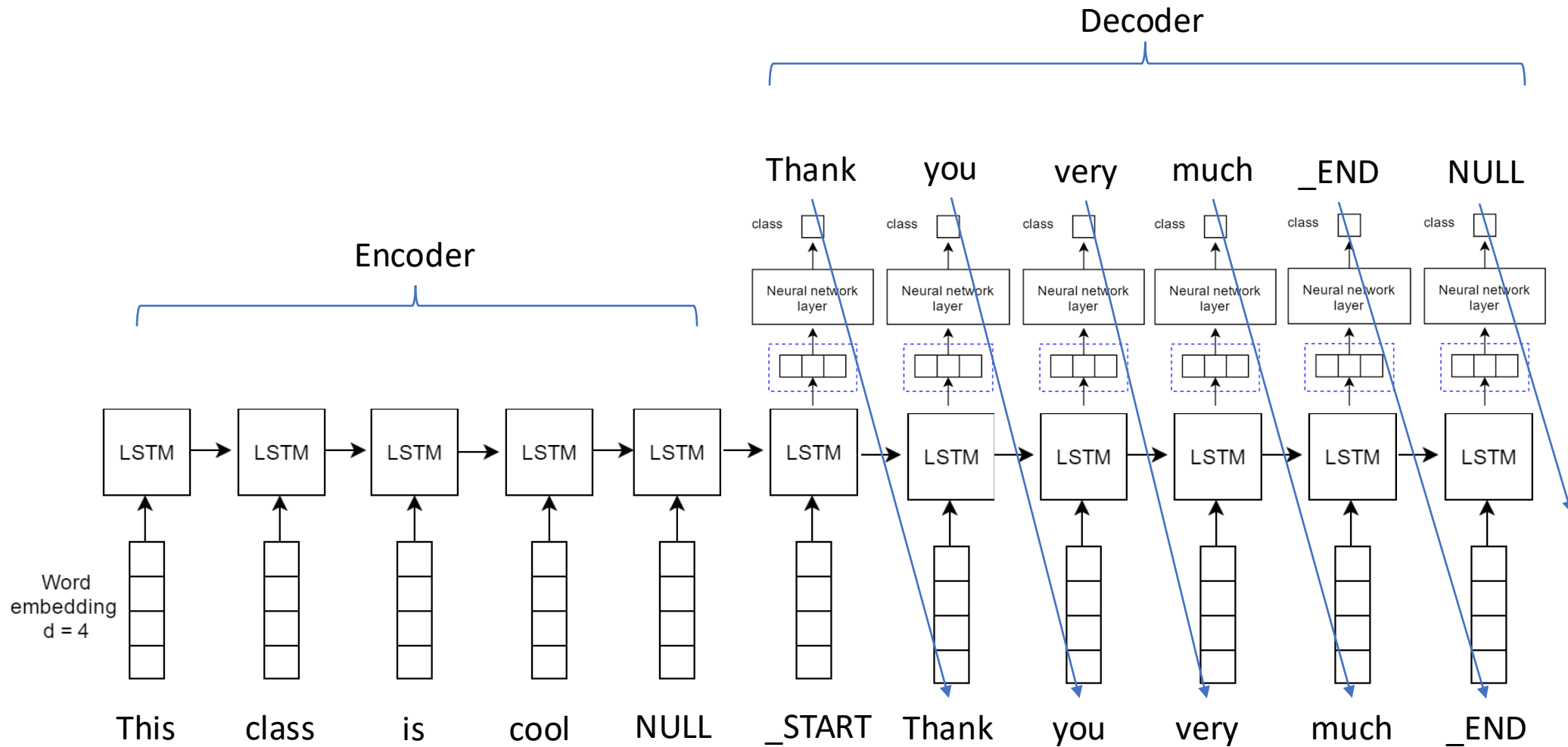
```
# build model
from keras.preprocessing import sequence
from keras.models import Sequential
from keras.layers import Dense, Embedding
from keras.layers import LSTM
max_features = total_words+1
maxlen = 80
batch_size = 32

# padding
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)

# model architecture
model = Sequential()
model.add(Embedding(max_features, 40, input_length=maxlen))
model.add(LSTM(32, dropout=0.20, recurrent_dropout=0))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

model.fit(x_train, y_train, batch_size=batch_size, epochs=3,
validation_data=(x_test, y_test))
```

Sequence-to-sequence model



Mini Group Project 4a

- Use the same reviews for restaurants and films dataset for the previous mini group projects.
- Build a training dataset with the first 400 restaurant reviews and the first 400 movie reviews (ID = [1:400, 501:900]) and use the rest as a test dataset.
- Use the same architecture as used in the demo to build a classifier for this dataset, setting the batch size to 20
- Increase the number of LSTM to 64
- Increase the epoch to 10
- Report the accuracies of the 3 architectures.

